Architectural Design Decision (ADR)

The Story of the Lost Developer

- What can we say about the image?
- "Why were these technologies chosen?"
- "Why was this standard chosen?"
  - There must be a good reason, but we don't know what it is.
- "The major problem with intellectual capital is that it has legs and walks home every day."

Lindvall, M., Rus, I., Jammalamadaka, R., & Thakker, R. (2001). Software Tools for Knowledge Management: A DACS State-of-the-art report. 2001. *Fraunhofer Center for Experimental Software Engineering Maryland and The University of Maryland: Maryland*.

The Story of the Lost Developer

- Architectural knowledge is subject to a phenomenon known as "knowledge vaporization."
  - Architectural knowledge is lost, inaccessible, or scattered over time
- Considering software architecture only as the structure of a project is no longer enough
- Architecture is also the set of decisions made, known as Architectural Decisions

Jansen, A., & Bosch, J. (2005, November). Software architecture as a set of architectural design decisions. In *5th Working IEEE/IFIP Conference on Software Architecture (WICSA'05)* (pp. 109-120). IEEE.

The Story of the Lost Developer

- Junior developer Silva is the newest hire at nutritional solutions company NutriLog.
- As a new hire, Junior is having a lot of trouble understanding what asynchronous communications are.
- He reads and rereads the NutriLog architecture diagrams, but just like asynchronous communications, there are several other concepts that he cannot understand in the diagram.

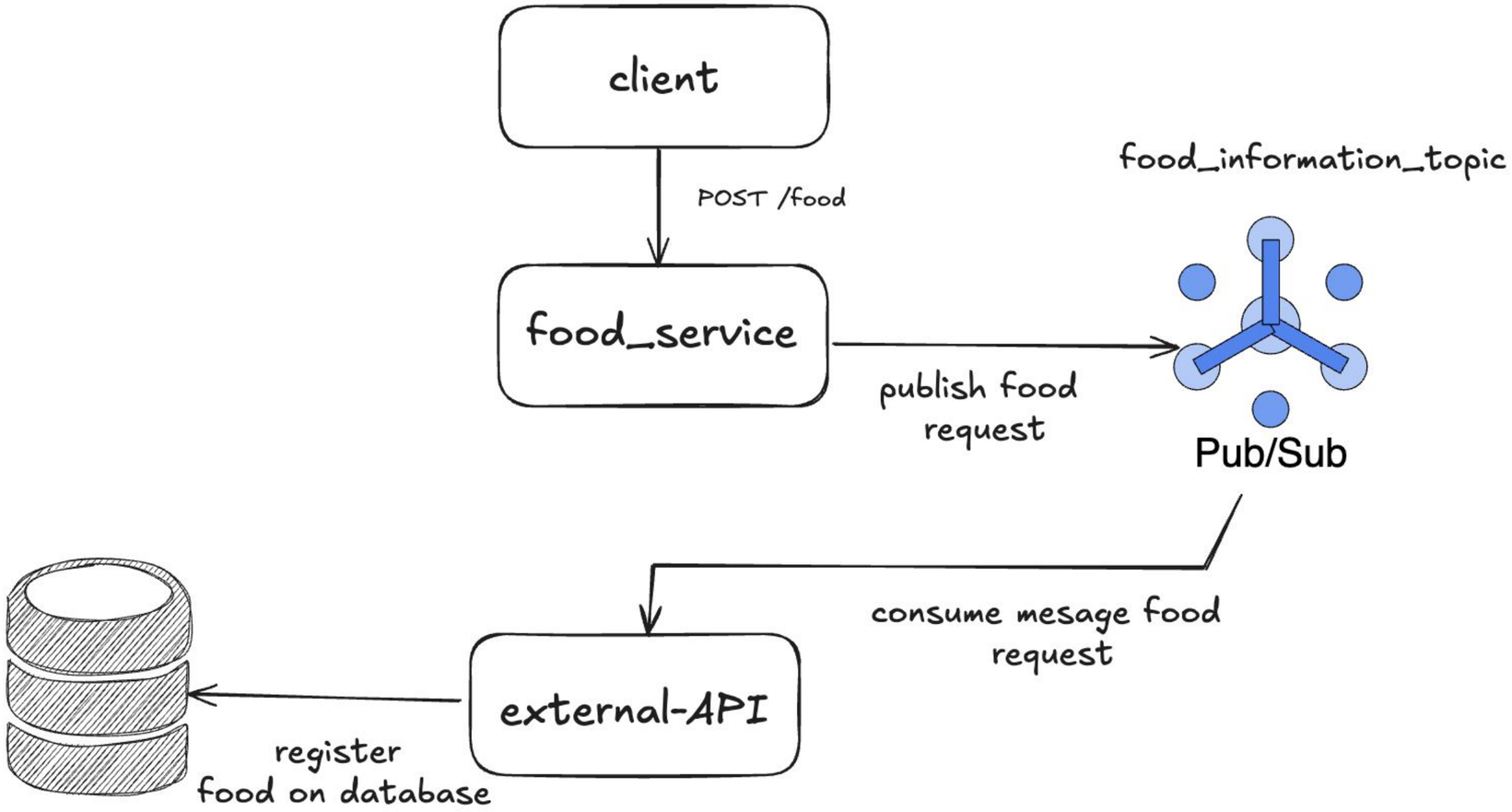
Jansen, A., & Bosch, J. (2005, November). Software architecture as a set of architectural design decisions. In *5th Working IEEE/IFIP Conference on Software Architecture (WICSA'05)* (pp. 109-120). IEEE.

The Story of the Lost Developer

- The team members are always too busy to teach how each part of the architecture works
- "Imagine if it had been written down somewhere why each component of this diagram exists."

Jansen, A., & Bosch, J. (2005, November). Software architecture as a set of architectural design decisions. In *5th Working IEEE/IFIP Conference on Software Architecture (WICSA'05)* (pp. 109-120). IEEE.

What are architectural decisions?



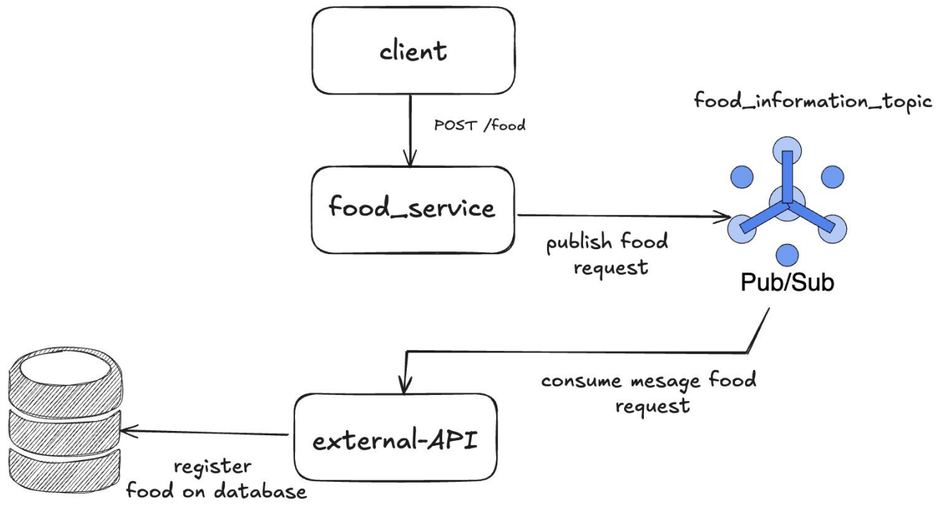
What are architectural decisions?

- According to the formal definition by [Jansen and Bosh]:
  - "A **description of the set of additions, subtractions, and modifications to the software architecture**,
  - the rationale and design rules, design constraints, and additional requirements that (partially) **fulfill one or more requirements** in a given architecture."

What are architectural decisions?

- Example:

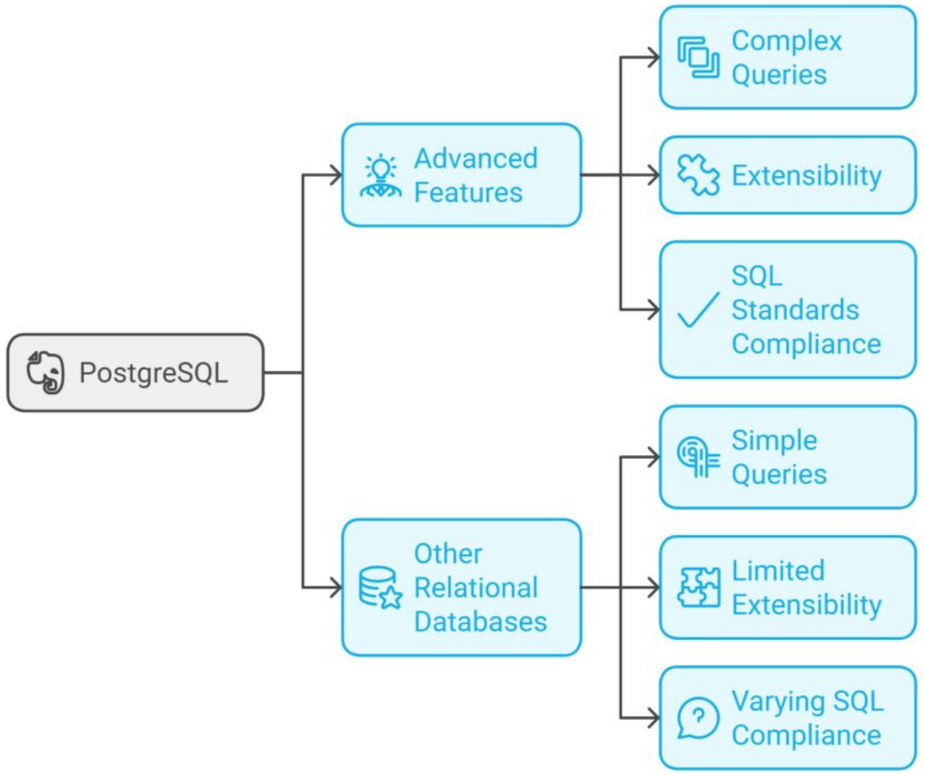
- **Problem:** If a communication error occurs between services, the food record is lost
- **Decision**: Make communication between food_service and the API that records food asynchronous
- **Tradeoffs:** Longer time to register new food items



What are architectural decisions?

- **Justification**: Contextualizes the current problem that will be solved with the decision
- **Rules:** These are the prerequisites for the project decision
- **Project constraints:** What cannot be included in the decision made
- **Consequences**: Impact that the decision made has on different parts of the project

Example: We need to use PostgresSQL because it is efficient in writing to the database and our system writes many times per second



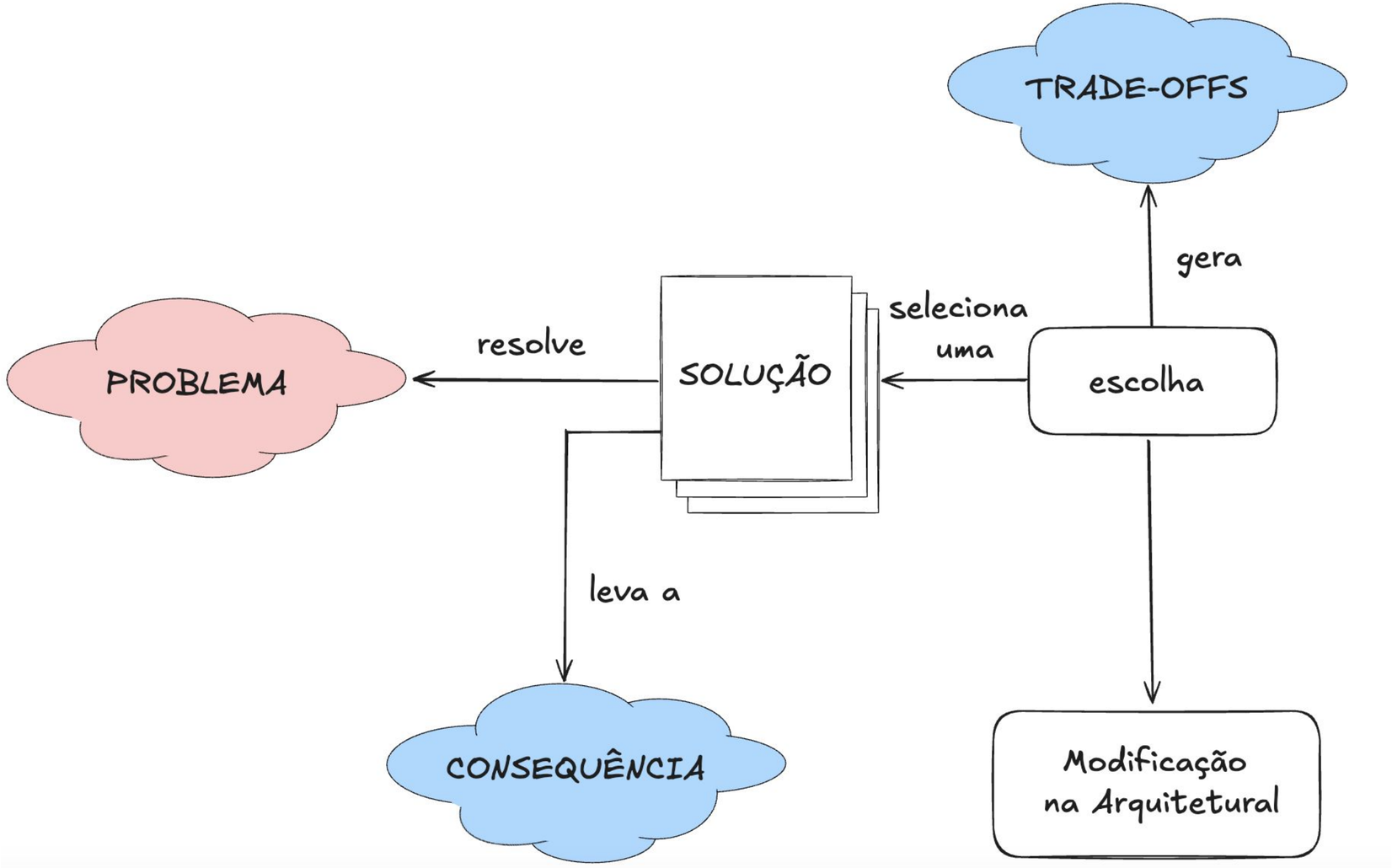
Problems with Not Documenting Decisions

- **Decisions are cross-cutting and interconnected**: Decisions are interconnected.
  - They affect various parts of the architecture.
  - Information about architecture decisions is fragmented, making it difficult to locate and change decisions.
  - Both effects increase the overall complexity of the software architecture

Problems with Not Documenting Decisions

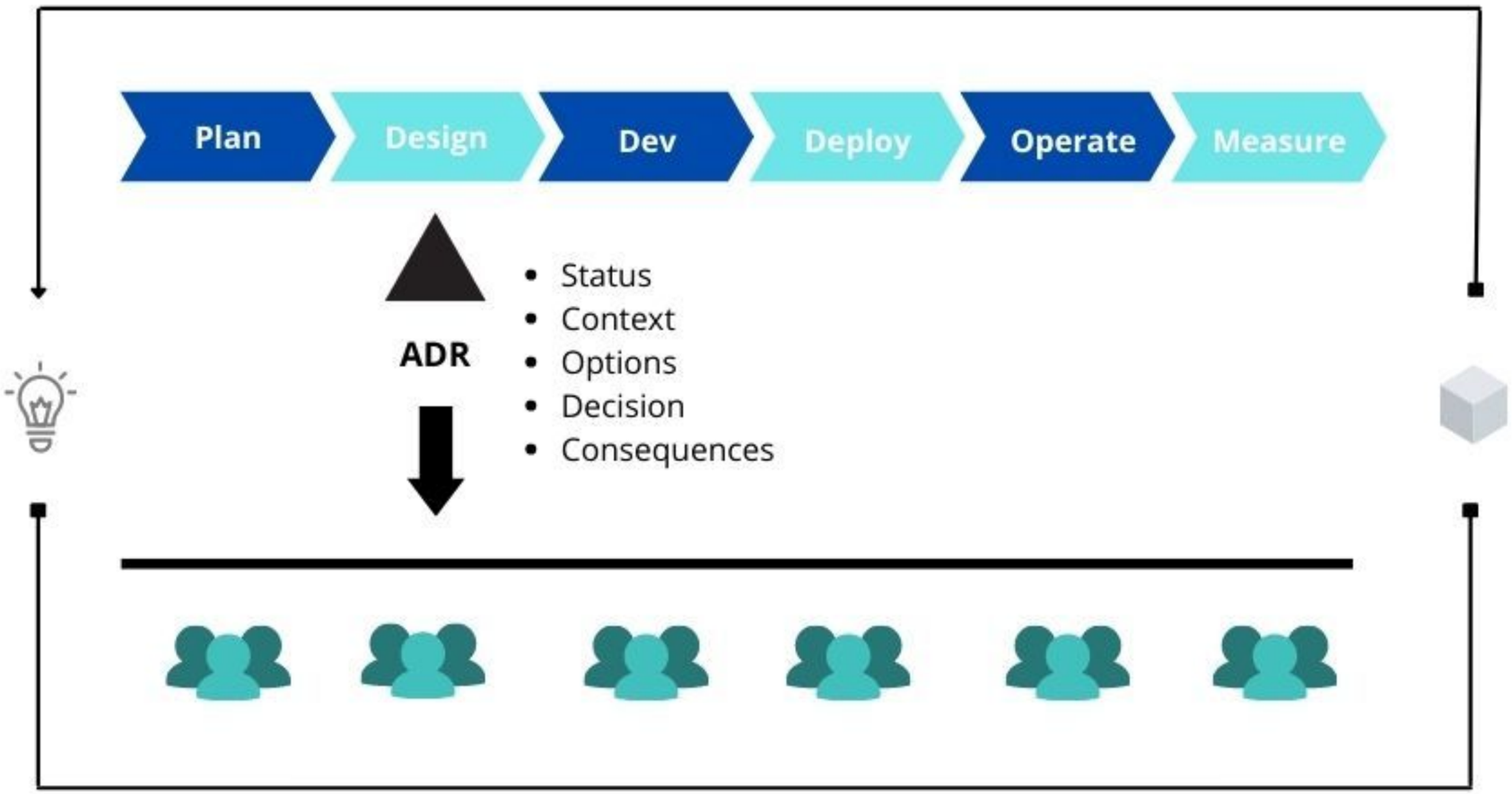
- **Design rules and restrictions are violated:**
  - During system evolution, designers can easily violate design rules and restrictions.
  - Violations of these rules and restrictions lead to problems associated with increased maintenance costs.
- **Obsolete design decisions are not removed:**
  - Modifications to the system are avoided to prevent fatigue
  - Tinkering with the architecture can have unexpected effects

Software Architecture as a set of decisions



An initial model for Architectural Decision proposed by [Jansen and Bosh] adapted

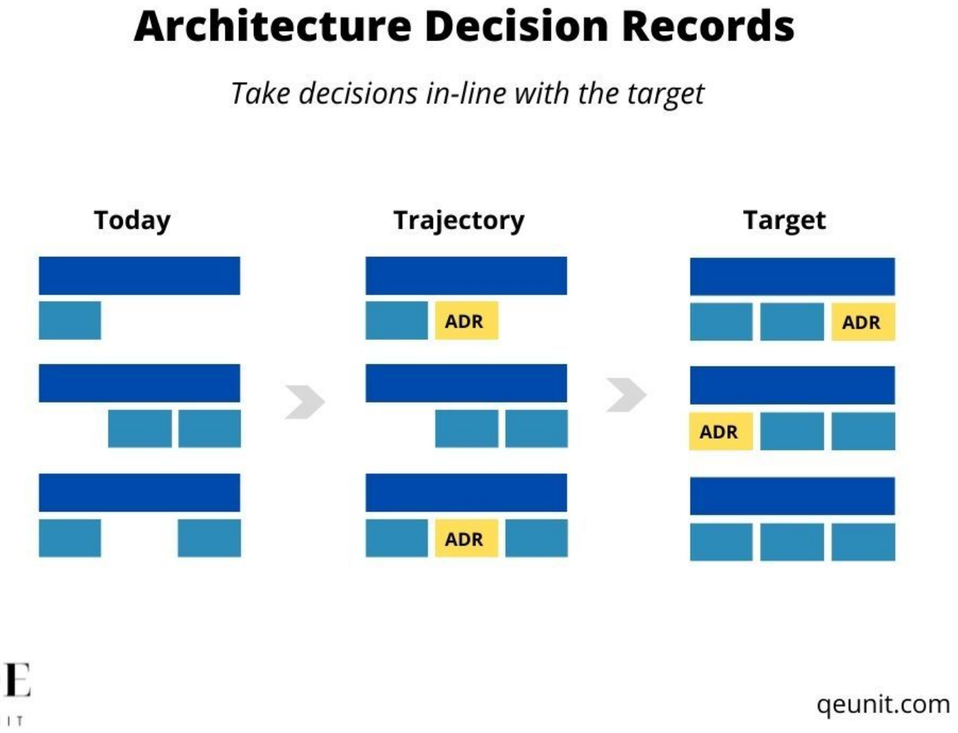
When to document Architectural Decisions



https://qeunit.com/blog/adrs-explicit-decisions-for-better-and-faster-software/

What Are ADRs (Architecture Decision Records)?

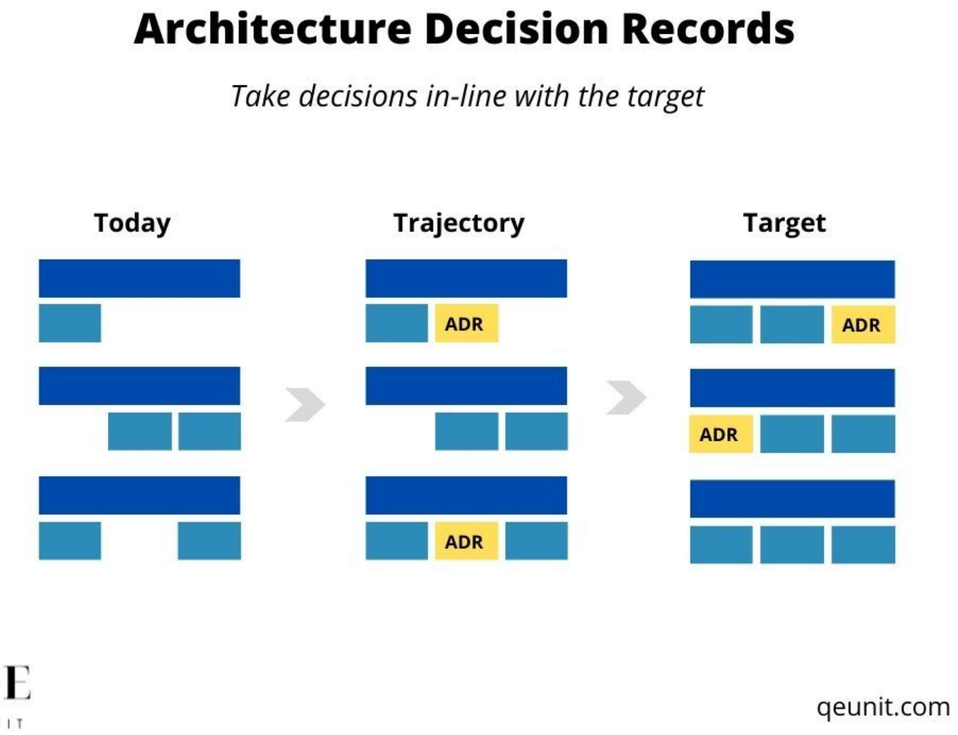
- **Focus on "Why":**
  - Record the reasons that led to the decision
  - It is not documenting the current state of the architecture
- **Message for the Future:**
  - They serve to explain to others (and to ourselves in the future) the reasoning behind the system architecture.



https://qeunit.com/blog/adrs-explicit-decisions-for-better-and-faster-software/

What are ADRs (Architecture Decision Records)?

- **But don't diagrams already solve this problem?**
  - While diagrams show what the architecture is,
  - ADRs explain why it was designed that way.



https://qeunit.com/blog/adrs-explicit-decisions-for-better-and-faster-software/

What are ADRs (Architecture Decision Records)?

- ADRs are generally:
  - A document created using a code/text management tool
  - Captures a single architectural decision.
  - Focus on context, decision, and consequences.
  - Impact the development lifecycle.

Practical Example of ADR

- **Title**: 001 - Using PostgreSQL as the primary relational database
- **Context**: Our application needs to persist user and product data, which are highly relational. We need ACID transactions to ensure order integrity. The team already has experience with SQL databases.
- **Decision**: We will adopt PostgreSQL as our standard relational database for all new services that require transactional consistency.
- **Consequences**:
  - **Positive**: Guaranteed data consistency (ACID). Mature and well-documented ecosystem. Leverages the team's existing knowledge, reducing the learning curve.
  - **Negative**: Less flexible for unstructured data. Requires a well-defined schema, which may slow down migrations in the future.

Why adopt ADRs?

Preserves Knowledge:

The "why" is recorded forever.

Accelerates Onboarding:

New developers understand the history of the project.

"Holy Wars":

Base discussions on decisions that have already been documented.

When to Use ADRs

Deserves ADR	Does Not Deserve ADR
Choice of Frameworks, Languages, and Databases	Variable names
Critical changes to system modules	Stylistic changes
Global System Definitions	Temporary changes

Decision Documentation Tools Architectural

- There are numerous tools available on the market
- Paid and open source tools
- .txt, .xml, .md, etc.

loqbooq

Log4brains
Architecture knowledge base

Time to Practice

- **Scenario**: E-commerce startup "VendeTudo" needs a user registration and authentication system. It must be secure, quick to implement, and scalable.
- Your Task (in groups):
  - Discuss the options:
  - Analyze the pros and cons (cost, time, security, maintenance).
  - Write an ADR to justify the decision (with context, decision, and consequence).

References

Martin, R. C. (2017). Clean architecture: a craftsman's guide to software structure and design. Prentice Hall Press.

Jansen, A., & Bosch, J. (2005, November). Software architecture as a set of architectural design decisions. In 5th Working IEEE/IFIP Conference on Software Architecture (WICSA'05) (pp. 109-120). IEEE.

Bhat, M., Tinnes, C., Shumaiev, K., Biesdorf, A., Hohenstein, U., & Matthes, F. (2019, March). ADeX: A Tool for Automatic Curation of Design Decision Knowledge for Architectural Decision Recommendations. In ICSA Companion (pp. 158-161).

Valente, M. T. (2020). Modern software engineering. Principles and practices for productive software development, 1(24).

https://ozimmer.ch/